Event Layer: Infra Improvements

Sprint: Week of Apr 13–19
Status: Shipped

What was missing before this week

The event layer could build clean `EventIntelligence` objects from data already passed to it, but nothing in the codebase could actually call it against the live database. There was no answer to the question:

> *"Give me the top 10 events right now."*

Beyond that, two reliability gaps existed:

1. Probability change fields (`probability_change_24h` , `probability_change_7d`) could silently return a number computed from snapshot history that was technically present but too sparse or stale to be meaningful.
2. The smoke script gave no visibility into how complete the output was — you could not tell at a glance whether 80% or 10% of events had valid 24h data.

What shipped

1. DB-backed event read entrypoint (`src/db/events.ts`)

Three public functions that agents and callers can import directly:

Function	What it returns
`getEventIntelligenceById(eventId)`	Single `EventIntelligence | null` for a Kalshi event ticker
`listEventIntelligenceByCategory(category, limit)`	Top events in a category, sorted by liquidity
`listTopEventIntelligence(limit)`	Top events across all active markets

Internally each function:

1. Pages through `markets` using `ORDER BY id` (indexed) to avoid statement timeouts
2. Fetches the last 8 days of snapshots in batches of 200 (covering both 24h and 7d windows)
3. Resolves historical resolution counts per cluster
4. Calls `clusterMarkets` + `buildEventIntelligence` from the existing pure-function layer
5. Sorts clusters by liquidity in memory — no `ORDER BY liquidity` at the DB level, which would cause a full-table scan

This is the missing bridge between the pure event-layer logic and anything that needs to actually call it.

Before: agents had no way to call the event layer against the DB.
After: `import { listTopEventIntelligence } from './db/events.js'` and done.

---

2. Honesty tests for sparse/stale probability changes

`probability_change_24h` and `probability_change_7d` are only useful if they represent a real measurement window. The proximity guard (`LOOKBACK_TOLERANCE_RATIO = 0.5`) was already in place, but the test suite did not explicitly verify the cases where it must return `null` .

Six new tests were added to `test/unit/event-intelligence.test.ts` :

Scenario	Expected
Only snapshot is 2 days old → 24h window	`null` (48h > 36h tolerance)
History only goes back 3 days → 7d window	`null` (3d < 3.5d tolerance floor)
Only same-day snapshots → 7d window	`null`
Snapshot 23h ago → 24h window	non-null (within 12h–36h band)
Snapshot exactly at outer tolerance → 24h window	`null` (strict > guard)
Snapshot 8 days ago → 7d window	non-null (within 3.5d–10.5d band)

These tests make the proximity guard's behavior explicit and prevent regressions if the tolerance constant is changed.

Before: a market with a single snapshot from 4 days ago would return `null` for `probability_change_24h` , but nothing in the test suite proved that.
After: there is a direct test for each sparse/stale boundary case.

---

3. Quality metrics in smoke output

`npm run event:show` now outputs a `quality_metrics` block alongside the events:

```
{
  "quality_metrics": {
    "total_events": 5,
    "pct_non_null_24h_change": 0.6,
    "pct_non_null_7d_change": 0.2,
    "pct_singleton_clusters": 0.4,
    "avg_related_markets_per_event": 2.3,
    "raw": {
      "non_null_24h": 3,
      "non_null_7d": 1,
      "singleton_clusters": 2,
      "total_related_markets": 11
    }
  },
  "events": [...]
}
```

Metric	What it tells you
`pct_non_null_24h_change`	How much of the 24h price-change signal is actually populated. Low value means snapshot cadence is too sparse or markets are too new.

`pct_non_null_7d_change`	Expected to be low for new deployments. Climbs as snapshot history accumulates past 7 days.
`pct_singleton_clusters`	How fragmented the clustering is. High value means Kalshi markets have inconsistent `event_id` and `series_id` coverage.
`avg_related_markets_per_event`	Average cluster size minus 1. Low value (near 0) means most events are singletons.

Before: you had to manually inspect the JSON to understand output quality.
After: one glance at `quality_metrics` shows the health of the event layer against real data.

How the pieces connect

```
DB (markets, market_snapshots, market_resolutions)
         │
         ▼
src/db/events.ts          ← NEW: fetches + assembles
         │
         ├── src/lib/event-clustering.ts   (clusterMarkets, selectPrimaryMarket)
         ├── src/lib/event-intelligence.ts (buildEventIntelligence,
computeProbabilityChange)
         └── src/lib/event-resolution-counts.ts (buildHistoricalResolutionCounts)
         │
         ▼
EventIntelligence[]       ← clean agent-readable output
```

The pure-function layer (`src/lib/`) remains unchanged and fully unit-tested. `src/db/events.ts` is the only place where DB I/O occurs for the event layer.

Test coverage

File	Tests
`test/unit/event-intelligence.test.ts`	+6 honesty tests (sparse/stale history)
`test/unit/event-clustering.test.ts`	no changes this week

Total suite: all tests passing, `npm run check` clean.

What is still weak

1. **No unit tests for `src/db/events.ts`** — the DB layer is integration-tested only via the smoke script. Adding Supabase mock tests would close this gap.
2. **`pct_non_null_7d_change` will be 0 for a while** — until the system has accumulated 7+ days of snapshot history for enough markets. This is expected and documented.
3. **`listTopEventIntelligence` pages all active markets into memory** — this scales fine today but will need a smarter strategy (pre-sorted index, or a materialized view) if the market count grows

past ~50k.